Embedded Opgave 1

Beskrivelse af software: Timer 1 LED-blink og UART-echo med bogstavkonvertering

Jacob Malthe Nielsen · ATmega328PB (16 MHz) · MPLAB XC8 3.00 · H4 Embedded

Overblik

Programmet består af tre moduler: `main.c`, `timer.c` og `uart.c`. Alt arbejde sker i interrupts, så hovedløkken i `main()` er tom.

- Timer 1 laver et interrupt hvert 500 ms, som toggler den grønne LED. LED'en blinker derfor med 1 Hz.
- UART0 modtager tegn fra PC'en, bytter om på store og små bogstaver og sender tegnet retur (echo).

Funktion	Ben	Forbundet til
UART0	PD0 (RXD0) / PD1 (TXD0)	PC via mEDBG's virtuelle COM-port (USB)
Grøn LED	PB3	Blinker med 1 Hz

UART'en kører 9600 baud, 8 databit, ingen paritet og 1 stopbit (8N1).

Opstart (main.c)

1. PB3 sættes som output til den grønne LED.
2. Timer 1 startes med et timeout på 500 ms: `Timer1_Init_ms(500)`.
3. UART'en initialiseres med `RS232Init()` (9600 baud, 8N1), og `ConvertReceivedChar` registreres som callback.
4. Modtage-interruptet og de globale interrupts (`sei()`) slås til, og `UART klar` sendes til terminalen.

Timer 1 (timer.c)

`Timer1_Init_ms(ms)` sætter Timer 1 op i CTC-mode (Clear Timer on Compare Match) med prescaler 256 og beregner `OCR1A` ud fra det ønskede antal millisekunder:

```
Ticks = (F_CPU / 256) × ms / 1000 − 1
      = 62 500 Hz × 0,5 s − 1 = 31 249    (ved 500 ms)
```

Når tælleren når `OCR1A`, starter den forfra fra 0, og `ISR(TIMER1_COMPA_vect)` kaldes. Interruptet toggler den grønne LED med `PORTB ^= (1 << GreenLedBit)`. LED'en skifter tilstand hvert 500 ms og blinker altså med 1 Hz.

UART (uart.c)

- `RS232Init()` sætter baudraten (`UBRR0 = F_CPU / (16 × 9600) − 1 = 103`), slår modtager og sender til og vælger 8 databit og 1 stopbit.
- `printf()` sender gennem UART'en, fordi `uart_putch` er sat som stdout.
- Hver gang der modtages et tegn, kalder `ISR(USART0_RX_vect)` den registrerede callback-funktion med en pointer til tegnet. Derefter sendes tegnet retur (echo) ved at skrive det i `UDR0`.
- Callback-mekanismen (`SetupUARTFunctionCallbackPointer`) betyder, at UART-driveren ikke ved, hvad applikationen gør med tegnene. Funktionen kan skiftes ud uden at ændre driveren.

Bogstavkonvertering (ConvertReceivedChar i main.c)

Store og små bogstaver i ASCII adskiller sig kun ved bit 5 (`0x20`), fx `'A' = 0x41` og `'a' = 0x61`. En XOR med `0x20` bytter dem derfor om i én linje kode:

```
if (((*ReceivedChar >= 0x41) && (*ReceivedChar <= 0x5D)) ||
    ((*ReceivedChar >= 0x61) && (*ReceivedChar <= 0x7D)))
{
    *ReceivedChar = *ReceivedChar ^ 0x20;
}
```

Intervallerne `0x41–0x5D` og `0x61–0x7D` dækker også `[ \ ]` ↔ `{ | }`, som i dansk 7-bit ASCII er **Æ Ø Å** ↔ **æ ø å**. Andre tegn sendes uændret retur.

Resultat

- Den grønne LED blinker med 1 Hz.
- En terminal på 9600 baud viser `UART klar` ved opstart.
- Alt, hvad der skrives i terminalen, kommer tilbage med store og små bogstaver byttet om, fx `Hej Verden` → `hEJ vERDEN`.